



HỌC PHẦN

# Lập trình mạng Network Programming

---

## CHƯƠNG 1

# Tổng quan về lập trình mạng

TS. Nguyễn Hoàng Hải

Khoa Toán - Tin • Đại học Sư phạm Đà Nẵng • 2026



# Mục tiêu Chương 1

## Khái niệm căn bản

Hiểu lập trình mạng là gì; tiến trình, thông điệp, địa chỉ IP, số hiệu cổng và socket.

## Giao thức mạng

Nắm vai trò của IP, TCP, UDP và các giao thức tầng ứng dụng dưới góc nhìn lập trình viên.

## Ngôn ngữ lập trình mạng

Biết các ngôn ngữ, thư viện socket phổ biến và lý do học phần chọn Python (đa nền tảng).

## Mô hình ứng dụng mạng

Phân biệt Client – Server, Peer-to-Peer, nhiều tầng; luồng hoạt động của ứng dụng TCP và UDP.

## Môi trường thực hành

Cài đặt Python 3.12 trở lên, VS Code và chạy chương trình mạng Python đầu tiên trên Windows hoặc macOS.

## Làm việc nhóm

Thảo luận, phân tích và trình bày — chuẩn bị cho bài tập nhóm xây dựng ứng dụng chat (CLO 5).



# Nội dung Chương 1

## 1.1 Các khái niệm căn bản

Slides 4–12

## 1.2 Các giao thức mạng

Slides 13–23

## 1.3 Ngôn ngữ lập trình mạng và môi trường Python

Slides 24–32

## 1.4 Mô hình hoạt động của ứng dụng mạng

Slides 33–42

# 1.1

## Các khái niệm căn bản

*Basic Concepts of Network Programming*



# 1.1 — Lập trình mạng là gì?

Lập trình mạng (network programming) là viết chương trình để các tiến trình chạy trên những máy tính khác nhau trao đổi dữ liệu với nhau thông qua mạng máy tính.

- ▶ **Đối tượng:** không lập trình cho router hay switch, mà cho ứng dụng chạy trên các máy đầu cuối (end system).
- ▶ **Bản chất:** gửi và nhận thông điệp (message) giữa hai tiến trình thông qua socket do hệ điều hành cung cấp.
- ▶ **Quen thuộc:** trình duyệt web, email, chat, trò chơi trực tuyến, xem video, thiết bị IoT — đều là ứng dụng mạng.
- ▶ **Trong học phần:** dùng Python xây dựng ứng dụng theo mô hình Client – Server, chạy trên Windows, macOS, Linux; đích đến là ứng dụng mạng hoàn chỉnh (chat, email, trình duyệt web,...)

## 1.1 — Ứng dụng mạng: các tiến trình trao đổi thông điệp



- ▶ **Tiến trình (process):** chương trình đang chạy. Hai tiến trình cùng máy giao tiếp bằng IPC của hệ điều hành; khác máy thì trao đổi thông điệp qua mạng.
- ▶ **Vị trí:** ứng dụng mạng nằm ở tầng ứng dụng; từ tầng giao vận trở xuống do hệ điều hành và thiết bị mạng đảm nhiệm.
- ▶ **Vai trò:** tiến trình Client chủ động khởi tạo liên lạc; tiến trình Server chờ được liên lạc và đáp lại.

## 1.1 — Định danh tiến trình: địa chỉ IP + số hiệu cổng

**192.168.1.10**

**Địa chỉ IP**

xác định máy tính trên mạng

**:8080**

**Số hiệu cổng (port)**

xác định tiến trình trên máy đó

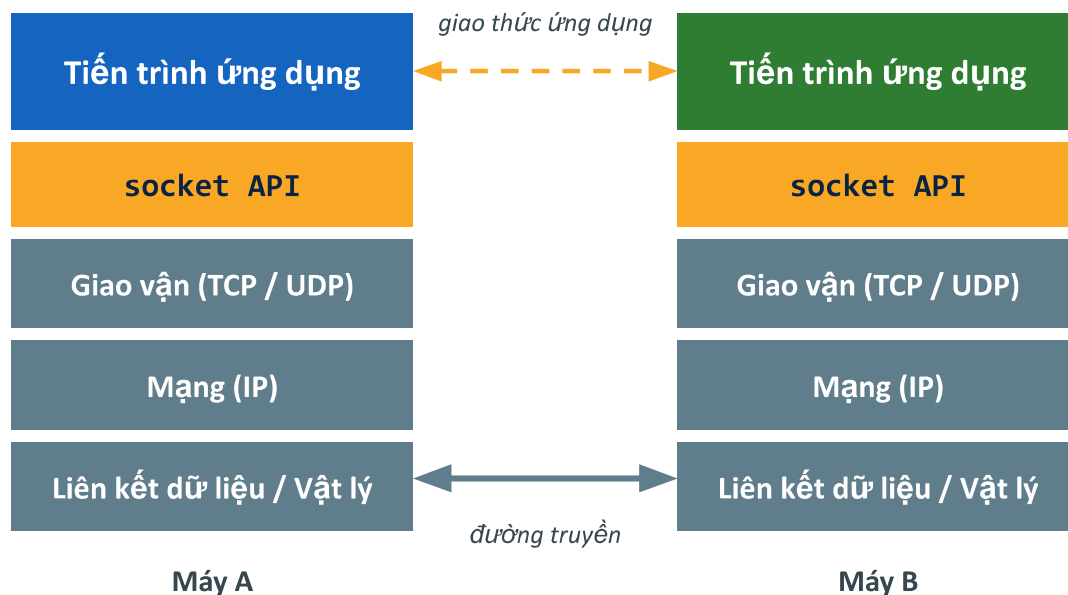
**192.168.1.10:8080**

**Endpoint (địa chỉ socket)**

IP + cổng — nơi thông điệp được gửi đến

- ▶ **Địa chỉ IP:** 32 bit (IPv4) hoặc 128 bit (IPv6) xác định máy tính; máy có thể có nhiều địa chỉ (nhiều card mạng, loopback 127.0.0.1).
- ▶ **Cổng:** số 16 bit (0 – 65535). Server dùng cổng cố định, biết trước (HTTP 80, DNS 53...); client thường được cấp cổng tạm 49152 – 65535.
- ▶ **Kết nối TCP:** được xác định duy nhất bởi bộ bốn (IP nguồn, cổng nguồn, IP đích, cổng đích).
- ▶ **Trong Python:** địa chỉ socket là bộ (host, port), ví dụ ("192.168.1.10", 8080); mô-đun ipaddress dùng để kiểm tra, xử lý địa chỉ IP (Chương 2).

# 1.1 — Socket: cửa giao tiếp giữa ứng dụng và mạng



- ▶ **Socket** là điểm cuối (endpoint) của một kênh liên lạc hai chiều, do hệ điều hành cung cấp cho ứng dụng qua một tập hàm gọi là socket API.
- ▶ **Phân công:** ứng dụng chỉ ghi dữ liệu vào socket và đọc từ socket; đóng gói, định tuyến, truyền lại... do hệ điều hành lo.
- ▶ **Hai loại chính:** stream socket (TCP) và datagram socket (UDP); ngoài ra có raw socket cho giao thức tầng thấp.
- ▶ **Kết luận:** lập trình mạng thực chất là lập trình với socket API — trong Python là mô-đun socket, nội dung của Chương 2.

■ lập trình viên điều khiển

■ hệ điều hành điều khiển

# 1.1 — Thông điệp, byte và mã hóa dữ liệu

- ▶ **Socket chỉ truyền byte:** chuỗi, số, đối tượng phải chuyển thành bytes trước khi gửi (encode / serialize) và chuyển ngược lại khi nhận; Python 3 tách bạch str và bytes.
- ▶ **Chuỗi văn bản:** phải chọn bảng mã; nên dùng UTF-8 (an toàn với tiếng Việt), tránh ASCII làm mất dấu.
- ▶ **Với TCP:** dữ liệu là luồng byte liên tục, không có ranh giới thông điệp → ứng dụng phải tự đánh dấu (ký tự kết thúc, tiền tố độ dài).
- ▶ **Với UDP:** mỗi lần gửi là một datagram trọn vẹn, có ranh giới rõ ràng nhưng kích thước hạn chế.
- ▶ **Thứ tự byte:** số nhiều byte cần thống nhất big-endian (network order) hay little-endian giữa hai bên.

```
Python - str, bytes, struct

# Chuỗi -> bytes (UTF-8)
data = "Xin chào!".encode("utf-8")

# bytes -> chuỗi
s = data.decode("utf-8")

# Số nguyên -> 4 byte (network order)
import struct
b = struct.pack("!I", 2026)
n, = struct.unpack("!I", b)
```

## 1.1 — Ứng dụng mạng cần gì từ tầng giao vận?

| Ứng dụng                 | Mất dữ liệu        | Băng thông           | Nhạy thời gian     | Giao vận          |
|--------------------------|--------------------|----------------------|--------------------|-------------------|
| Truyền file, email, web  | Không chấp nhận    | Co giãn              | Không              | TCP               |
| Chat văn bản (học phần)  | Không chấp nhận    | Thấp                 | Không              | TCP               |
| Gọi thoại / video (VoIP) | Chấp nhận một phần | Vài trăm kbps – Mbps | Rất nhạy (~100 ms) | UDP               |
| Trò chơi trực tuyến      | Chấp nhận một phần | Thấp – trung bình    | Nhạy (vài chục ms) | UDP (hoặc TCP)    |
| Video theo yêu cầu       | Chấp nhận một phần | Cao                  | Vài giây           | TCP/HTTP hoặc UDP |

**Bốn yêu cầu: độ tin cậy, thông lượng, thời gian, bảo mật. TCP đáp ứng tin cậy, UDP đáp ứng tốc độ; bảo mật cần thêm TLS/SSL.**

# 1.1 — Thuật ngữ cần nắm

## Host / End system

Máy đầu cuối chạy ứng dụng mạng: PC, điện thoại, máy chủ. Được định danh bằng địa chỉ IP.

## Server & Client

Server chờ và phục vụ yêu cầu; client chủ động gửi yêu cầu. Là vai trò của tiến trình.

## Thông điệp (message)

Đơn vị dữ liệu ứng dụng gửi / nhận; được đóng gói thành segment, packet, frame khi đi xuống các tầng.

## Băng thông & độ trễ

Lượng dữ liệu truyền mỗi giây (bps) và thời gian gói tin đi từ nguồn đến đích (ms).

## Kết nối (connection)

Kênh logic hai chiều giữa hai socket TCP, có thiết lập và đóng. UDP không có kết nối.

## Blocking / Non-blocking

Lời gọi chờ đến khi hoàn tất hay trả về ngay — nền tảng của Chương 6 và 7.



## Tóm tắt 1.1 — Các khái niệm căn bản

1. Lập trình mạng là viết chương trình để các tiến trình trên những máy khác nhau trao đổi thông điệp qua mạng.
2. Ứng dụng mạng chạy trên máy đầu cuối, ở tầng ứng dụng; từ tầng giao vận trở xuống do hệ điều hành đảm nhiệm.
3. Tiến trình được định danh bằng địa chỉ IP (máy) và số hiệu cổng (ứng dụng); cặp (IP, cổng) là endpoint.
4. Socket là cửa giao tiếp giữa ứng dụng và mạng — lập trình mạng thực chất là lập trình với socket API.
5. Socket chỉ truyền bytes: Python 3 tách bạch str và bytes, cần encode / decode UTF-8 và tự xác định ranh giới thông điệp khi dùng TCP.
6. Mỗi ứng dụng có yêu cầu riêng về tin cậy, thông lượng, thời gian → chọn TCP hay UDP cho phù hợp.

# 1.2

## Các giao thức mạng

*Network Protocols*



## 1.2 — Giao thức mạng: "ngôn ngữ chung" của hai bên

Giao thức (protocol) quy định khuôn dạng và thứ tự các thông điệp trao đổi giữa các thực thể mạng, cùng hành động cần thực hiện khi gửi hoặc nhận một thông điệp.

### Cú pháp (Syntax)

Cấu trúc, định dạng thông điệp: các trường, độ dài, thứ tự byte, ký tự phân cách.

### Ngữ nghĩa (Semantics)

Ý nghĩa từng trường và hành động tương ứng: yêu cầu, trả lời, báo lỗi, xác nhận.

### Thời gian (Timing)

Thứ tự trao đổi, tốc độ, thời gian chờ (timeout) và quy tắc truyền lại.

Với lập trình viên: giao thức chuẩn (HTTP, SMTP, DNS...) đã được đặc tả trong các RFC; ứng dụng riêng thì phải tự thiết kế giao thức.

## 1.2 — Chồng giao thức TCP/IP dưới góc nhìn lập trình viên

| Tầng           | Giao thức tiêu biểu                           | Ai xử lý?            | Lập trình viên làm gì?                          |
|----------------|-----------------------------------------------|----------------------|-------------------------------------------------|
| Ứng dụng       | HTTP, FTP, SMTP, DNS, giao thức tự định nghĩa | Ứng dụng             | Viết toàn bộ: định dạng thông điệp, logic xử lý |
| Giao vận       | TCP, UDP                                      | Hệ điều hành         | Chọn TCP hay UDP, mở socket, gửi / nhận         |
| Internet       | IP, ICMP, ARP                                 | Hệ điều hành, router | Chỉ cung cấp địa chỉ IP đích                    |
| Truy nhập mạng | Ethernet, Wi-Fi                               | Card mạng, driver    | Không can thiệp                                 |

Ranh giới giữa tầng ứng dụng và tầng giao vận chính là socket API — nơi lập trình viên làm việc.

## 1.2 — IP: giao thức chuyển gói nền tảng

- ▶ **Nhiệm vụ:** đánh địa chỉ và định tuyến gói tin (packet) từ máy nguồn đến máy đích qua nhiều mạng trung gian.
- ▶ **Best-effort:** không bảo đảm gói đến nơi, đúng thứ tự hay không trùng lặp — việc bảo đảm tin cậy giao cho TCP.
- ▶ **IPv4 và IPv6:** IPv4 32 bit (192.168.1.10) vẫn phổ biến; IPv6 128 bit (2001:db8::1) đang thay thế dần.
- ▶ **Lập trình viên:** không thao tác trực tiếp với IP header; chỉ cung cấp địa chỉ đích cho socket.
- ▶ **Trong Python:** địa chỉ viết dạng chuỗi: "0.0.0.0" hoặc "" là mọi card mạng, "127.0.0.1" là loopback, ":::" cho IPv6; mô-đun `ipaddress` để kiểm tra, tính mạng con.

```
Python — ipaddress

import ipaddress
ip = ipaddress.ip_address(
    "192.168.1.10")
print(ip.version, ip.is_private)
# 4 True
net = ipaddress.ip_network(
    "192.168.1.0/24")
print(net.num_addresses, ip in net)
# 256 True
```

## 1.2 — TCP: hướng kết nối, tin cậy, luồng byte

- ▶ **Kết nối:** bắt tay ba bước (SYN, SYN-ACK, ACK) trước khi truyền; đóng kết nối bằng cờ FIN.
- ▶ **Tin cậy:** đánh số thứ tự, báo nhận (ACK), truyền lại khi mất; có điều khiển luồng và điều khiển tắc nghẽn.

### ✓ Ưu điểm

Dữ liệu đến đủ, đúng thứ tự, không trùng; lập trình đơn giản: đọc / ghi như luồng (NetworkStream).

### ✗ Hạn chế

Tốn chi phí thiết lập kết nối; header 20 byte; độ trễ cao hơn UDP; không giữ ranh giới thông điệp.



Trong Python: `socket.socket(AF_INET, SOCK_STREAM)`; tiện ích `socket.create_server()`, `create_connection()` — Chương 3 và Chương 5.

## 1.2 — UDP: không kết nối, nhanh, datagram

- ▶ **Không bắt tay:** gửi ngay datagram tới (IP, cổng) đích; mỗi datagram độc lập, header chỉ 8 byte.
- ▶ **Không bảo đảm:** gói có thể mất, đến sai thứ tự hoặc trùng lặp; ứng dụng tự xử lý nếu cần.

### ✓ Ưu điểm

Nhanh, chi phí thấp; giữ nguyên ranh giới thông điệp; hỗ trợ broadcast và multicast trong LAN.

### ✗ Hạn chế

Phải tự kiểm soát mất gói và thứ tự; datagram lớn dễ bị phân mảnh (nên giữ dưới 1.472 byte).

💡 Trong Python: `socket.socket(AF_INET, SOCK_DGRAM)` với `sendto()` / `recvfrom()`; `socketserver.UDPServer` — Chương 4 và Chương 5.

## 1.2 — Chọn TCP hay UDP cho ứng dụng?

### Chọn TCP khi...

- ▶ **Không được mất dữ liệu:** file, email, web, chat, giao dịch.
- ▶ **Khối lượng lớn,** thứ tự dữ liệu quan trọng.
- ▶ **Muốn lập trình đơn giản:** đọc / ghi như luồng.
- ▶ **Trong học phần:** chat server, truyền file.

### Chọn UDP khi...

- ▶ **Ưu tiên tốc độ,** chấp nhận mất vài gói: thoại, video trực tiếp, game.
- ▶ **Thông điệp ngắn,** hỏi – đáp một lần: DNS, DHCP, SNMP.
- ▶ **Cần broadcast / multicast:** tìm thiết bị, thông báo trong LAN.
- ▶ **Trong học phần:** chat UDP, tìm server trong LAN.

Nhiều ứng dụng hiện đại dùng cả hai; QUIC (nền của HTTP/3) xây dựng tin cậy ngay trên UDP.

## 1.2 — Giao thức tầng ứng dụng tiêu biểu

| Giao thức    | Chức năng                  | Giao vận          | Cổng                          | Đặc tả          |
|--------------|----------------------------|-------------------|-------------------------------|-----------------|
| HTTP / HTTPS | Truyền trang web, API REST | TCP (HTTP/3: UDP) | 80 / 443                      | RFC 9110 – 9114 |
| FTP          | Truyền tập tin             | TCP               | 21 (điều khiển), 20 (dữ liệu) | RFC 959         |
| SMTP         | Gửi thư điện tử            | TCP               | 25, 587                       | RFC 5321        |
| POP3 / IMAP  | Nhận thư điện tử           | TCP               | 110 / 143 (995 / 993 TLS)     | RFC 1939 / 9051 |
| DNS          | Phân giải tên miền         | UDP (TCP khi dài) | 53                            | RFC 1034 / 1035 |
| DHCP         | Cấp phát địa chỉ IP        | UDP               | 67 / 68                       | RFC 2131        |
| SSH / Telnet | Điều khiển máy từ xa       | TCP               | 22 / 23                       | RFC 4251 / 854  |

**Các cổng này do IANA quy định (well-known ports 0 – 1023); ứng dụng tự viết nên dùng cổng từ 1024 trở lên.**

## 1.2 — Ví dụ giao thức ứng dụng: HTTP là văn bản thuần

```
telnet www.example.com 80

# Client -> Server (request)
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: MyClient/1.0
Connection: close
                                <- dòng trống
# Server -> Client (response)
HTTP/1.1 200 OK
Date: Fri, 04 Sep 2026 08:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 1256
                                <- dòng trống
<!DOCTYPE html>
<html> ... </html>
```

- ▶ **Yêu cầu (request):** dòng lệnh (phương thức, đường dẫn, phiên bản) + các header + dòng trống + thân (nếu có).
- ▶ **Trả lời (response):** dòng trạng thái (200 OK, 404 Not Found...) + header + dòng trống + thân.
- ▶ **Văn bản thuần:** mọi thứ là ký tự, mỗi dòng kết thúc bằng CRLF (\r\n) → có thể thử bằng telnet hoặc tự viết client.
- ▶ **Thực hành:** dùng socket TCP trong Python gửi request GET thử công và in response ra màn hình (Chương 3).

## 1.2 — Tự thiết kế giao thức cho ứng dụng của bạn

- ▶ **Liệt kê thông điệp:** các loại yêu cầu / trả lời — đăng nhập, gửi tin, danh sách người dùng, thoát, báo lỗi.
- ▶ **Chọn định dạng:** văn bản có dấu phân cách (dễ gỡ lỗi), JSON (linh hoạt) hay nhị phân (gọn, nhanh).
- ▶ **Đánh dấu ranh giới:** kết thúc bằng "\n" hoặc tiền tố độ dài (4 byte) — bắt buộc khi dùng TCP.
- ▶ **Quy định trạng thái:** trình tự hợp lệ (phải LOGIN trước khi MSG), mã lỗi, thời gian chờ.
- ▶ **Viết tài liệu trước khi code:** cả nhóm cùng tuân thủ một đặc tả — yêu cầu của bài tập nhóm.

Giao thức chat (văn bản, phân cách |)

```
C: LOGIN|nam\n
S: OK|Chào nam\n
C: MSG|all|Xin chào cả lớp\n
S: MSG|nam|Xin chào cả lớp\n
C: LIST\n
S: USERS|nam,lan,hoa\n
C: MSG|lan|Bạn ới\n
C: QUIT\n
S: BYE\n
```



## Tóm tắt 1.2 — Các giao thức mạng

1. Giao thức gồm cú pháp, ngữ nghĩa và thời gian; giao thức chuẩn được đặc tả trong các RFC.
2. Lập trình viên làm việc ở tầng ứng dụng và chọn TCP / UDP qua socket; từ IP trở xuống do hệ điều hành đảm nhiệm.
3. IP chuyển gói best-effort; TCP bổ sung kết nối, tin cậy, luồng byte; UDP nhanh, datagram, không bảo đảm.
4. Chọn TCP khi không được mất dữ liệu; chọn UDP khi ưu tiên tốc độ, thông điệp ngắn hoặc cần broadcast.
5. HTTP, FTP, SMTP, DNS, DHCP... là giao thức ứng dụng chuẩn với cổng cố định; HTTP là văn bản thuần.
6. Ứng dụng riêng phải tự thiết kế giao thức: loại thông điệp, định dạng, ranh giới, trạng thái và mã lỗi.

# 1.3

## Ngôn ngữ lập trình mạng và môi trường Python

*Network Programming Languages & the Python Environment*



## 1.3 — Socket API: chuẩn chung của mọi ngôn ngữ

Berkeley sockets  
BSD 4.2 UNIX (C)

1983

Winsock 1.0  
cho Windows

1992

Python 3.0  
tách bạch bytes / str

2008

Python 3.14  
phiên bản mới nhất

2025

1991

Python ra đời  
(Guido van Rossum)

1996

Java 1.0  
java.net.Socket

2015

Python 3.5  
async / await (asyncio)

Dù dùng ngôn ngữ nào, các bước socket → bind → listen → accept → send / recv → close đều giống nhau, vì cùng gọi socket API của hệ điều hành.

## 1.3 — Các ngôn ngữ lập trình mạng phổ biến

### Python — ngôn ngữ chính của học phần

socket, socketserver, asyncio, threading; tkinter cho giao diện. Đa nền tảng, mã ngắn; kiểm thử, tự động hóa, khoa học dữ liệu.

### C / C++

Berkeley sockets, Winsock, Boost.Asio. Hiệu năng cao; lập trình hệ thống, nhúng, game engine.

### Java

java.net, java.nio, Netty. Ứng dụng doanh nghiệp, máy chủ lớn, Android.

### C# / .NET

System.Net.Sockets, TcpClient, async/await. Ứng dụng Windows, dịch vụ, game (Unity).

### JavaScript / Node.js

net, http, WebSocket, Socket.IO. Web thời gian thực, dịch vụ nhiều kết nối I/O.

### Go / Rust

net (Go), tokio (Rust). Dịch vụ đám mây, hiệu năng cao, an toàn bộ nhớ, song song.

## 1.3 — Cùng một việc, nhiều ngôn ngữ: mở kết nối TCP

### C (Berkeley sockets)

```
int s = socket(AF_INET,  
              SOCK_STREAM, 0);  
connect(s, (struct sockaddr*)  
        &addr, sizeof(addr));  
send(s, buf, len, 0);  
close(s);
```

### Java

```
Socket s = new Socket(  
    "127.0.0.1", 8080);  
OutputStream out =  
    s.getOutputStream();  
out.write(data);  
s.close();
```

### Python

```
import socket  
s = socket.create_connection(  
    ("127.0.0.1", 8080))  
s.sendall(b"Hello")  
s.close()
```

→ Khác cú pháp, giống bản chất: tạo socket, kết nối tới (IP, cổng), gửi dữ liệu, đóng — cùng gọi socket API của hệ điều hành. Python gọn nhất.

## 1.3 — Vì sao học phần chọn Python?

- ▶ **Đa nền tảng thật sự:** cùng một tệp .py chạy trên Windows, macOS, Linux; không phụ thuộc Visual Studio hay Windows Forms.
- ▶ **Sát với socket API gốc:** mô-đun socket phản ánh trực tiếp Berkeley sockets — học Python xong đọc mã C, Java, C# đều hiểu.
- ▶ **"Batteries included":** socketserver, selectors, asyncio, threading, struct, json, http, tkinter có sẵn trong thư viện chuẩn, không cần cài thêm.
- ▶ **Mã ngắn, demo nhanh:** server echo chỉ khoảng 10 dòng; dùng REPL để thử từng lệnh socket ngay tại lớp.
- ▶ **Được dùng rộng rãi:** kiểm thử mạng, an ninh mạng, tự động hóa, khoa học dữ liệu; giáo trình Kurose & Ross cũng dạy socket bằng Python.

Học phần dùng Python 3.12 trở lên với VS Code hoặc PyCharm Community. GIL của Python không ảnh hưởng ứng dụng mạng thiên về chờ I/O (Chương 7).

## 1.3 — Các mô-đun Python dùng trong học phần

| Mô-đun                               | Hàm / lớp chính                                                                                     | Dùng để                                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>socket</b>                        | socket(), bind, listen, accept, connect, send / recv, sendto / recvfrom, gethostbyname, getaddrinfo | Lập trình socket TCP / UDP, phân giải DNS (C2 – C4)    |
| <b>struct, json</b>                  | pack / unpack, dumps / loads                                                                        | Định dạng và ranh giới thông điệp (C3)                 |
| <b>socketserver</b>                  | TCPServer, UDPServer, ThreadingMixIn, StreamRequestHandler                                          | Khung server có sẵn, phục vụ nhiều client (C5)         |
| <b>http, urllib, smtplib, ftplib</b> | http.client, http.server, urlopen                                                                   | Giao thức tầng ứng dụng chuẩn (C5)                     |
| <b>selectors, asyncio</b>            | DefaultSelector; start_server, open_connection, StreamReader / Writer                               | Socket non-blocking, lập trình bất đồng bộ (C6)        |
| <b>threading, concurrent.futures</b> | Thread, Lock, Queue; ThreadPoolExecutor                                                             | Server đa luồng, nhận tin song song với giao diện (C7) |
| <b>tkinter</b>                       | Tk, Text, Entry, Button, after()                                                                    | Giao diện ứng dụng chat đa nền tảng (C7)               |

**Tất cả nằm trong thư viện chuẩn: chỉ cần import, không cần pip install.**

## 1.3 — Chương trình Python đầu tiên: tên máy và địa chỉ IP

```
hello_net.py - python hello_net.py

import socket

host = socket.gethostname()
print("Máy:", host)

infos = socket.getaddrinfo(host, None)
for family, _, _, _, addr in infos:
    print(family.name, addr[0])

print("IPv4:", socket.gethostbyname(host))
# Kết quả (ví dụ):
# Máy: LAB-PC07
# AF_INET6 fe80::1c2d:9a3f%12
# AF_INET 192.168.1.10
# IPv4: 192.168.1.10
```

- ▶ **gethostname():** trả về tên máy đang chạy chương trình.
- ▶ **getaddrinfo(host, None):** hỏi hệ điều hành / DNS, trả về danh sách bộ 5 phần tử (family, type, proto, canonname, sockaddr).
- ▶ **family.name:** AF\_INET là IPv4, AF\_INET6 là IPv6; sockaddr là bộ (địa chỉ, cổng, ...).
- ▶ **Thực hành:** lưu thành hello\_net.py, chạy trong VS Code hoặc terminal và so sánh với ipconfig / ifconfig.

## 1.3 — Công cụ hỗ trợ lập trình và kiểm thử mạng

### Python 3.12 trở lên

Cài từ python.org, Homebrew (macOS) hoặc Microsoft Store; kiểm tra bằng `python --version`.

### VS Code / PyCharm

Viết, chạy, gỡ lỗi (breakpoint, watch); VS Code cần extension Python; PyCharm Community miễn phí; có REPL tích hợp.

### Wireshark

Bắt và phân tích gói tin: nhìn thấy bắt tay ba bước, nội dung thông điệp, lỗi giao thức.

### ipconfig, netstat, ping

Xem địa chỉ IP, cổng đang lắng nghe (`netstat -an`), kiểm tra kết nối; macOS dùng `ifconfig`, `lsof -i`.

### ncat (nc) / telnet

"Client vận năng": kết nối tay tới server TCP / UDP để thử giao thức trước khi viết client.

### curl / Postman / http.server

Gửi yêu cầu HTTP, xem response; `python -m http.server` dựng web server thử chỉ bằng một lệnh.



## Tóm tắt 1.3 — Ngôn ngữ lập trình mạng & Python

1. Socket API ra đời từ BSD UNIX (1983); Winsock, Java, .NET, Python... đều kế thừa — bản chất giống nhau.
2. Mọi ngôn ngữ phổ biến đều lập trình mạng được; học phần chọn Python vì đa nền tảng, sát socket gốc, thư viện chuẩn đầy đủ, mã ngắn.
3. Mô-đun chính: socket (C2 – C4), struct / json (C3), socketserver (C5), selectors / asyncio (C6), threading và tkinter (C7).
4. Python 3 tách bạch str và bytes: mọi dữ liệu đi qua socket đều là bytes, cần encode / decode UTF-8.
5. Chương trình đầu tiên: socket.gethostname() và socket.getaddrinfo() lấy tên máy và địa chỉ IP.
6. Công cụ: Python 3.12+, VS Code / PyCharm, Wireshark, ipconfig / ifconfig / netstat / ping, ncat, curl.

# 1.4

## Mô hình hoạt động của ứng dụng mạng

*Network Application Architectures & Models*



## 1.4 — Ba kiến trúc ứng dụng mạng

### Client – Server

- ▶ **Server:** luôn bật, địa chỉ IP cố định, chờ yêu cầu.
- ▶ **Client:** chủ động kết nối; các client không liên lạc trực tiếp với nhau.
- ▶ **Ví dụ:** web, email, CSDL, ứng dụng chat.

### Peer-to-Peer (P2P)

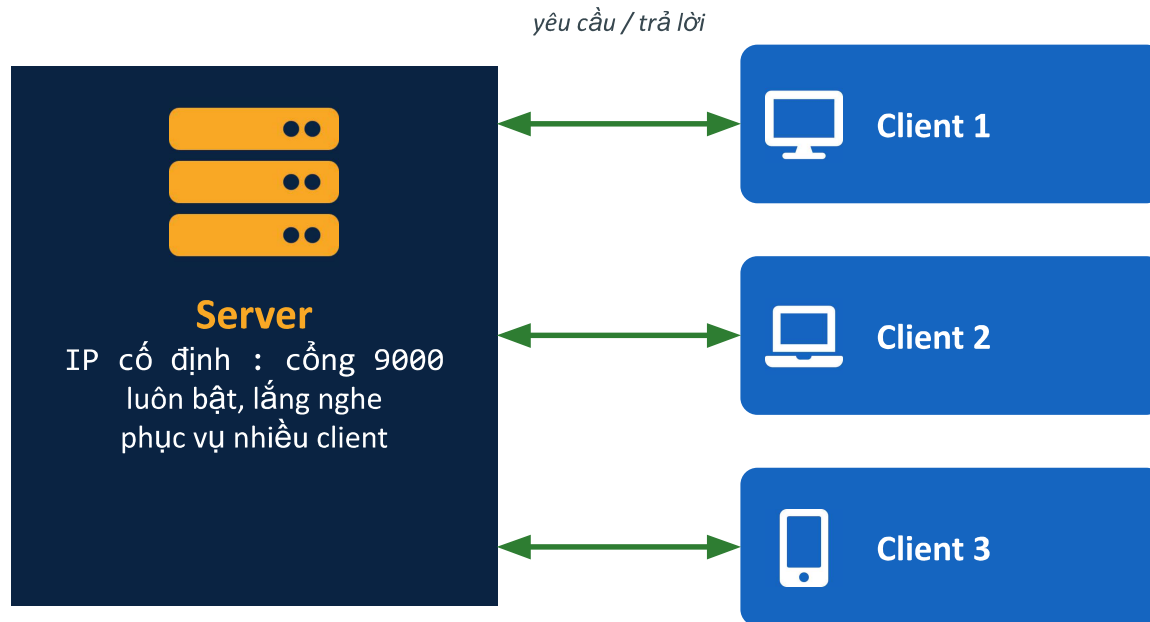
- ▶ **Bình đẳng:** mỗi máy vừa là client vừa là server.
- ▶ **Phân tán:** không cần máy chủ trung tâm, tự mở rộng khi thêm máy.
- ▶ **Ví dụ:** BitTorrent, blockchain, chia sẻ file.

### Lai (Hybrid)

- ▶ **Server trung tâm:** đăng nhập, danh bạ, tìm đối tác.
- ▶ **Dữ liệu:** trao đổi trực tiếp giữa các peer.
- ▶ **Ví dụ:** gọi video WebRTC, game online (server ghép trận + P2P).

Học phần xây dựng ứng dụng theo mô hình Client – Server: đơn giản, dễ quản lý và là nền của hai mô hình còn lại.

## 1.4 — Mô hình Client – Server



- **Server:** tiến trình thụ động — mở socket, gắn với cổng cố định, lắng nghe và phục vụ nhiều client.
- **Client:** tiến trình chủ động — kết nối đến IP:cổng của server, gửi yêu cầu, nhận trả lời.
- **Vai trò của tiến trình:** một máy có thể vừa chạy server vừa chạy client — thử nghiệm với 127.0.0.1.
- **Điều kiện:** server phải khởi động trước; client cần biết địa chỉ server (cấu hình, DNS hoặc broadcast tìm kiếm).

## 1.4 — Luồng hoạt động của ứng dụng TCP



Trong Python: `socket.create_server((host, port)) = socket + bind + listen`; `conn, addr = s.accept()`; `socket.create_connection((host, port)) = socket + connect`.

## 1.4 — Luồng hoạt động của ứng dụng UDP



Không có connect / accept: mỗi datagram tự mang địa chỉ. Trong Python: `data, addr = s.recvfrom(1024)` và `s.sendto(data, addr)`.

## 1.4 — Server tuần tự và server đồng thời

### Server tuần tự (iterative)

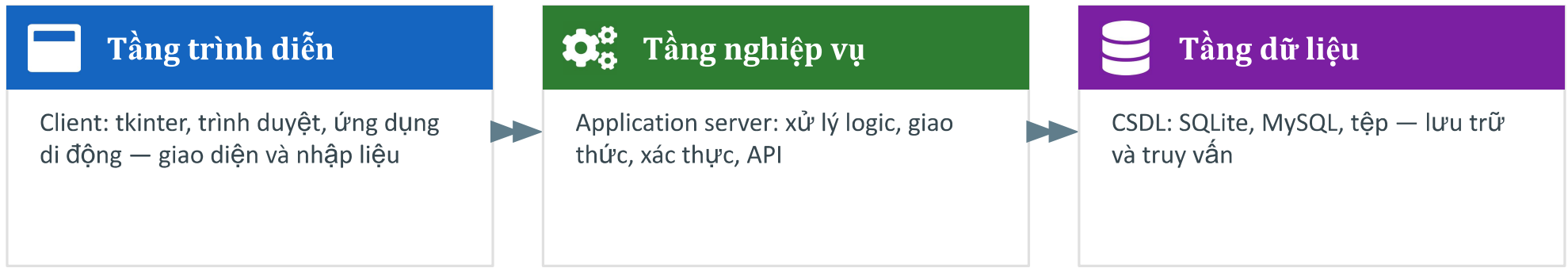
- ▶ **Cách làm:** phục vụ xong một client mới accept client kế tiếp.
- ▶ **Ưu:** đơn giản; hợp với dịch vụ hỏi – đáp ngắn (echo, time, DNS).
- ▶ **Nhược:** một client chậm hoặc treo sẽ chặn tất cả client khác.

### Server đồng thời (concurrent)

- ▶ **Đa luồng:** mỗi client một threading.Thread, hoặc ThreadingTCPServer, ThreadPoolExecutor — Chương 7.
- ▶ **Bất đồng bộ:** asyncio với async / await, hoặc socket non-blocking + selectors — Chương 6.
- ▶ **Ưu / nhược:** phục vụ hàng trăm client cùng lúc; cần đồng bộ hóa dữ liệu dùng chung.

Ứng dụng chat nhiều người dùng bắt buộc phải là server đồng thời: vừa nhận tin của người này vừa phát cho người khác.

## 1.4 — Kiến trúc nhiều tầng (2-tier, 3-tier, n-tier)



- ▶ **2 tầng:** client nối thẳng tới CSDL hoặc một server duy nhất — đơn giản nhưng khó mở rộng, khó bảo mật.
- ▶ **3 tầng:** tách logic ra server ứng dụng — client "mỏng", dễ thay đổi và mở rộng từng tầng.
- ▶ **n tầng / microservices:** nhiều dịch vụ nhỏ nói chuyện với nhau qua mạng (HTTP, gRPC, hàng đợi thông điệp).
- ▶ **Ứng dụng chat học phần:** client tkinter ↔ chat server ↔ (tùy chọn) SQLite lưu lịch sử — kiến trúc 3 tầng thu nhỏ.

## 1.4 — Kiểu tương tác: hỏi – đáp, đẩy và luồng

### Hỏi – đáp (Request – Response)

Client hỏi, server trả lời rồi xong.  
Ví dụ: HTTP, DNS. Đơn giản, nhưng server không thể chủ động gửi khi có sự kiện.

### Đẩy (Push / Publish – Subscribe)

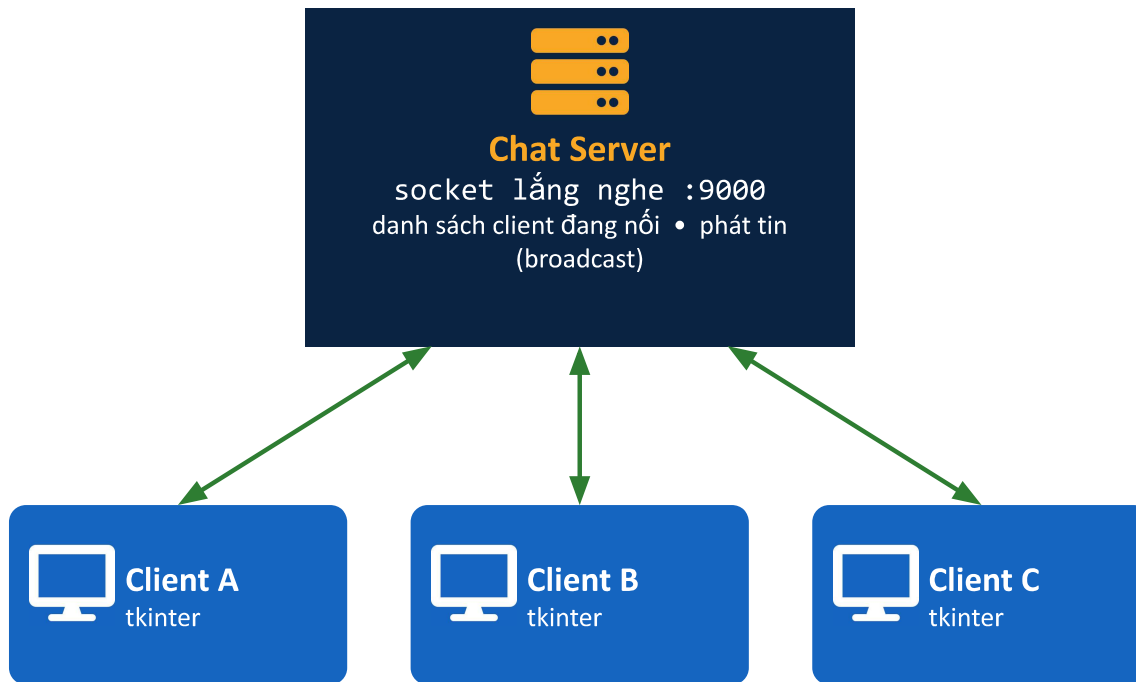
Server chủ động gửi cho client khi có sự kiện: tin nhắn mới, thông báo. Cần kết nối được duy trì (TCP, WebSocket).

### Luồng (Streaming)

Dữ liệu chảy liên tục theo thời gian: video, âm thanh, cảm biến. Thường dùng UDP / RTP hoặc HTTP theo đoạn.

Ứng dụng chat kết hợp hỏi – đáp (đăng nhập, danh sách) và đẩy (tin nhắn mới) → cần kết nối TCP duy trì và một luồng riêng để nhận tin.

## 1.4 — Ví dụ: kiến trúc ứng dụng chat của học phần



MSG|all|Xin chào → server → phát tới mọi client

- **Server:** socket lắng nghe cổng 9000; mỗi client một thread nhận; giữ danh sách kết nối để phát tin cho mọi người.
- **Client:** tkinter — kết nối, gửi tin theo giao thức; một thread riêng nhận tin, cập nhật giao diện qua `after()` để không bị treo.
- **Giao thức:** văn bản, phân cách "|", kết thúc "\n" — đã phác thảo ở mục 1.2.
- **Lộ trình:** C2 socket & DNS → C3 TCP → C5 lớp hỗ trợ → C6 – C7 bất đồng bộ, đa luồng → sản phẩm cuối kỳ.

## Tóm tắt 1.4 — Mô hình hoạt động của ứng dụng mạng

1. Ba kiến trúc: Client – Server (server luôn bật, client chủ động), Peer-to-Peer (bình đẳng) và lai (kết hợp).
2. Client và server là vai trò của tiến trình; một máy có thể chạy cả hai và thử nghiệm qua 127.0.0.1.
3. TCP: server socket → bind → listen → accept → recv / send → close; client socket → connect → send / recv → close.
4. UDP: không connect / accept; sendto và recvfrom kèm địa chỉ trong từng datagram.
5. Server tuần tự phục vụ từng client; server đồng thời dùng đa luồng hoặc bất đồng bộ để phục vụ nhiều client.
6. Kiến trúc 2 / 3 / n tầng tách trình diễn – nghiệp vụ – dữ liệu; ứng dụng chat học phần là Client – Server đồng thời.

# Tổng kết Chương 1 — Tổng quan về lập trình mạng

## 1.1 Khái niệm

Lập trình mạng; tiến trình & thông điệp; IP + cổng = endpoint; socket; byte & mã hóa; yêu cầu dịch vụ.

## 1.2 Giao thức

Cú pháp – ngữ nghĩa – thời gian; IP, TCP, UDP; chọn TCP hay UDP; giao thức ứng dụng; tự thiết kế giao thức.

## 1.3 Ngôn ngữ

Socket API chung; các ngôn ngữ; vì sao Python; mô-đun socket, socketserver, asyncio, threading; chương trình đầu tiên; công cụ.

## 1.4 Mô hình

Client – Server, P2P, lai; luồng TCP / UDP; server tuần tự & đồng thời; kiến trúc nhiều tầng; ứng dụng chat.

→ Tiếp theo — Chương 2: Lập trình Socket với Python (mô-đun socket và phân giải DNS)



## Thực hành tuần 1 & chuẩn bị cho Chương 2

### Tại lớp — 1 tiết thực hành

1. Cài Python 3.12 trở lên (python.org, Homebrew hoặc Microsoft Store) và VS Code với extension Python.
2. Viết hello\_net.py lấy tên máy và địa chỉ IP, chạy bằng python hello\_net.py; so sánh với ipconfig / ifconfig.
3. Dùng netstat -an xem các cổng đang lắng nghe; ping 127.0.0.1 và ping máy bên cạnh.
4. Nâng cao: chạy python -m http.server 8000, mở trình duyệt xem; dùng ncát gửi "GET / HTTP/1.1" và quan sát response.

### Ở nhà — lớp học đảo ngược

- Đọc Rhodes & Goerzen, Foundations of Python Network Programming: Chương 1, 2; Kurose & Ross: mục 2.7 Socket Programming.
- Nhóm 5 sinh viên: chọn đề tài, phác thảo đặc tả giao thức thông điệp (dạng văn bản).
- Thảo luận: vì sao chat dùng TCP còn gọi video dùng UDP? Ứng dụng chat của nhóm có cần server đồng thời không?



# Cảm ơn!

## Chương 1 — Tổng quan về lập trình mạng

---

### Tài liệu tham khảo

- [1] B. Rhodes, J. Goerzen — Foundations of Python Network Programming, 3rd ed., Apress, 2014 (giáo trình) — Chương 1, 2.
- [2] Đỗ Thị Hương Lan, Trần Hồng Nghi — Tài liệu hướng dẫn thực hành Lập trình mạng căn bản, NXB ĐHQG TP.HCM, 2024.
- [3] J. Kurose, K. Ross — Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021 — mục 2.7 Socket Programming.
- [4] W. R. Stevens, B. Fenner, A. Rudoff — UNIX Network Programming, Vol. 1, 3rd ed., Addison-Wesley, 2004.
- [5] Python Software Foundation — The Python Standard Library (socket, socketserver, selectors, asyncio, threading) — [docs.python.org](https://docs.python.org).
- [6] RFC 791 (IP), RFC 9293 (TCP), RFC 768 (UDP), RFC 9110 (HTTP) — [ietf.org](https://ietf.org).